



École Polytechnique

BACHELOR THESIS IN COMPUTER SCIENCE

On Formalizing a Bijection between Linear Lambda Terms and Rooted Trivalent Maps

Author:

Sacha Gregoire, École Polytechnique

Advisor:

Noam Zeilberger, École Polytechnique

Academic year 2025/2026

Abstract

This thesis reports a mechanized formalization of the informal bijection between linear lambda terms and rooted trivalent maps. Building on the combinatorial construction recorded in "Linear lambda terms as invariants of rooted trivalent maps" by Zeilberger and the original combinatorial enumeration work of Bodini, Gardy and Jacquot, the development begins to implement the correspondence inside the Rocq proof environment using the MathComp support library.

The formalization proceeds in three stages. First, core data structures and invariants were encoded: an inductive representation of open linear lambda terms parameterized by the number of free variables, combinatorial maps parametrized by the number of boundaries, and a reachability predicate to enforce connectivity. Second, executable coercions from terms to maps and the inverse building blocks were constructed and their local correctness properties were mechanically proven. Third, structural decomposition lemmas were developed to identify when a map piece corresponds to an application or to an abstraction and to reduce maps by removing the root and its adjacent vertex.

The mechanization verifies a broad suite of routine but essential lemmas, and provides a reusable Rocq record for rooted trivalent map isomorphisms. The development does not complete the full, general inverse proofs: the final inverse theorems (injectivity and surjectivity of the two coercions over arbitrary dart-types) remain open due to time constraints. The thesis documents these remaining proof obligations, isolates the key technical bottlenecks, and supplies a clear roadmap for completing the bijectivity proof.

Contents

1	Introduction	4
2	Preliminaries	4
2.1	Linear Lambda Calculus	4
2.2	Rooted Trivalent Maps	4
2.3	From Linear Lambda Terms to Rooted Trivalent Maps	5
2.4	From Rooted Trivalent Maps to Linear Lambda Terms	6
3	Formalizing Definitions	6
3.1	Linear Lambda Terms	6
3.2	Reachability and Transitivity	7
3.3	Rooted Trivalent Maps	8
3.4	Isomorphisms of Rooted Trivalent Maps	8
4	Formalizing Coercions From Linear Lambda Terms to Rooted Trivalent Maps	9
4.1	Variables	10
4.2	Application of two Rooted Trivalent Maps	10
4.3	Abstraction of a Rooted Trivalent Map	12
4.4	Final Construction	14
5	Formalizing Coercions From Rooted Trivalent Maps to Linear Lambda Terms	14
5.1	Proving Maps have at least two Darts	14
5.2	Maps with two Darts are isomorphic to the trivial map	14
5.3	Maps with more than two Darts	15
5.4	A Slight Adaptation of Reachability	16
5.5	Abstraction Case	18
5.6	Partitioning T for the Application Case	19
6	Conclusion	20
7	References	22
A	Appendix	23

1 Introduction

The goal of this thesis is to formalize a bijection between two well-known combinatorial objects: linear lambda terms and rooted trivalent maps. Informally, every linear lambda term determines a rooted trivalent map with boundary darts corresponding to free variables, and conversely every rooted trivalent map determines a linear lambda term. The formalization proceeds in two parts. First, represent the two families of objects inside Rocq using the MathComp library, giving executable representations and decision procedures for the local tests needed in the bijection. Second, implement the two coercions that convert terms into maps and that convert maps back into terms, and prove that these coercions are inverses. The central deliverable of this thesis is a mechanized development that formalizes the definitions, the inductive constructions, and the main lemmas used in the informal bijection. The deliverable stops short of a fully mechanized inverse proof in the most general representation, and the concluding sections explain how to continue the work of the thesis.

2 Preliminaries

The goal of this thesis is to work on formalizing a bijection between linear lambda terms and rooted trivalent maps in Rocq [2]. In this section, we formally define the two primary mathematical objects of our study: linear lambda terms and rooted trivalent maps. All definitions and coercion methods are adapted from those in Zeilberger's paper "Linear lambda terms as invariants of rooted trivalent maps" [3].

2.1 Linear Lambda Calculus

Lambda calculus terms are constructed from variables using the operations of application and abstraction. For example, x , $x(y)$, and $\lambda x.y$ are all lambda terms. An abstraction $\lambda x[t]$ is said to bind the occurrences of x within the subterm t , and any variable that is not bound by an abstraction is said to be free.

Linear lambda terms are lambda terms in which every variable is used exactly once. We consider open linear lambda terms, which can have free variables.

Note that we study lambda terms up to alpha equivalence; we consider two lambda terms to be equivalent if they are the same up to renaming variables.

2.2 Rooted Trivalent Maps

Abstractly, a trivalent map is an embedding of a graph in which each vertex is trivalent (has three neighbors), except for some number of boundary vertices, which are univalent (have one neighbor). A rooted trivalent map is a trivalent map with a distinguished boundary vertex called the root. Notably, every rooted trivalent map must have at least one boundary.

To define maps rigorously, we use the combinatorial approach based on permutations of darts (half-edges).

Definition 2.1 (Combinatorial Map). A combinatorial map is a connected finite graph embedded in a closed orientable surface such that the edges do not intersect, and the faces are homeomorphic to open disks.

Formally, a map is a triple $M = (T, v, e)$ where:

- T is a finite set of elements called *half-edges* (or darts).
- v is a permutation on T . The disjoint cycles of v correspond to the vertices of the map, and the cyclic order of half-edges represents their counterclockwise order around the vertex.
- e is a fixed-point-free involution on T . The 2-cycles of e correspond to the edges of the map, joining two half-edges.

The group generated by the permutations v and e must act transitively on T to ensure the map is connected.

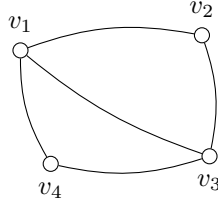


Figure 1: Example of a map.

Definition 2.2 (Trivalent Map). A combinatorial map $M = (T, v, e)$ is called *trivalent* if every non-boundary vertex has a degree of exactly 3. In the permutation representation, this means that every cycle in the permutation v has a length of either 1 if the corresponding vertex is a boundary vertex, or 3 if the corresponding vertex is a trivalent vertex.

Definition 2.3 (Rooted Map). A map is *rooted* if one half-edge $r \in T$ is distinguished as the root. The choice of a root eliminates non-trivial automorphisms, rigidifying the structure. In a rooted trivalent map, the vertex associated with the root half-edge is called the root vertex. We require that the root vertex is a boundary vertex.

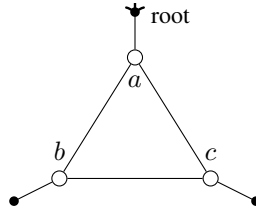


Figure 2: Example of a rooted trivalent map

2.3 From Linear Lambda Terms to Rooted Trivalent Maps

To turn a lambda term into a rooted trivalent map, we begin by drawing a tree representing the underlying structure of applications and abstractions, turn each application and abstraction into a trivalent vertex, and add extra edges connecting each bound variable to its corresponding abstraction.

You end up with a hanging edge for the output of the lambda term and hanging edges for each free variable of the lambda term. At the end of each of these hanging edges, we add univalent vertices. The vertex corresponding to the output of the lambda term is the root, and all other vertices are boundary vertices. This procedure results in a rooted trivalent map with the same amount of non-root boundary vertices as the lambda term has free variables.

For more information on this procedure, an interested reader can read Zeilberger's explanation [\[3\]](#).

For example, for the identity term $\lambda x.x$, we begin with the tree on the left, turning the hanging edge on top into the root and adding a link from the other hanging edge to the bound variable to obtain the rooted trivalent map on the right.

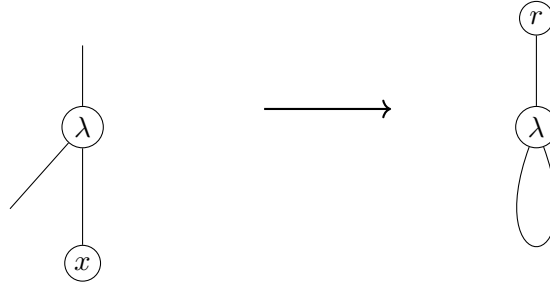


Figure 3: Example of the correspondence for the linear lambda term $\lambda x. x$.

2.4 From Rooted Trivalent Maps to Linear Lambda Terms

To turn a rooted trivalent map into a lambda term, we need to be able to decide if each vertex is an application node or an abstraction node. The decision procedure is as follows: for the trivalent vertex incident to the root, if removing that vertex and the root disconnects the graph, the vertex corresponds to an application node, whereas if the graph remains connected, the vertex corresponds to an abstraction node. Iterating this procedure until you reach the trivial map tells you which kind of node every vertex corresponds to. This procedure results in a linear lambda term with the same number of free variables as the number of non-root boundary vertices in the rooted trivalent map.



Figure 4: The trivial map consisting of a single edge. The root is the top vertex r .

As each linear lambda term corresponds to a rooted trivalent map and this process is reversible, the two sets of objects witness a bijection.

3 Formalizing Definitions

Before beginning to formalize the bijection in Rocq, we had to pick a representation for each of the objects. While choosing a representation seems like a tame task, it was actually one of the most important, challenging, and revisited parts of the thesis, and the representations we ended up with are completely different than the original ones.

In addition to the Rocq programming language, we also used the MathComp library [1] to aid with the definitions and proofs.

3.1 Linear Lambda Terms

We chose to define linear lambda terms by making them take as input the number of free variables.

Source Code 1: Linear Lambda Terms

```

1 Inductive llambda : nat -> Type :=
2 | LVar      : llambda 1
3 | LApp {p q} : llambda p -> llambda q -> llambda (p+q)
4 | LAbs {k}   : 'I_(S k) -> llambda (S k) -> llambda k
5 .

```

We state that a variable is a linear lambda term with 1 free variable, a lambda application takes linear lambda terms with p and q free variables respectively and maps them to a linear lambda term with $p + q$ free variables, and a lambda abstraction takes a linear lambda term with $k + 1$ free variables, along with an index in $[0, \dots, k]$ that represents which of the $k + 1$ free variables is being abstracted, and maps these to a linear lambda term with k free variables.

3.2 Reachability and Transitivity

To define rooted trivalent maps, we need a way to ensure that the graph is connected. We chose to do this by enforcing transitivity of darts through v and e .

Source Code 2: Reachability and Transitivity

```

1 Inductive reachable {T : finType} (v e : T -> T) (y : T) : T -> Prop :=
2 | reach_r : (reachable v e y) y
3 | reach_v : forall x, (reachable v e y) x -> (reachable v e y) (v x)
4 | reach_e : forall x, (reachable v e y) x -> (reachable v e y) (e x).
5
6 Definition transitive {T : finType} (v e : T -> T) (r : T) : Prop :=
7   forall x, (reachable v e r) x.

```

Here, T represents the set of darts, while v and e will come from the definition of the rooted trivalent map. We state that any vertex y is reachable from itself, and that if x is reachable from y , then so are $v(x)$ and $e(x)$. We define transitivity as every dart being reachable from a specific dart, which in our case will always be the root dart.

We also wish to show that reachability is an equivalence relation for proof purposes. By the definition of reachability above, we get that it is a reflexive relation. It remains to show that the relation is transitive and symmetric.

Source Code 3: Transitivity and Symmetry of Reachability

```

1 Lemma reachtrans {T : finType} (y : T) v e (x z : T) :
2   reachable v e x y -> reachable v e y z -> reachable v e x z.
3
4 Lemma reachsym {T : finType} v e
5   (v3 : forall x, v (v (v x)) = x) (e2 : forall x, e (e x) = x) (x y : T) :
6   reachable v e x y -> reachable v e y x.

```

The proof of transitivity consists of a simple induction on the reachability of z from y . The proof of symmetry consists of a simple induction on the reachability of y from x , and also requires the $v3$ and $e2$ hypotheses which will be seen in the definition of a rooted trivalent map, as symmetry does not necessarily hold in directed graphs.

3.3 Rooted Trivalent Maps

Now that we have defined what it means to be transitive, we are able to define rooted trivalent maps. Similarly to how lambda terms take as input the number of free variables, maps take as input the number of non-root boundary vertices.

Source Code 4: Rooted Trivalent Maps

```

1 Record rtmap (k : nat) : Type := {                                (* Rooted trivalent maps *)
2   T      : finType;                                                (* Dart type *)
3   v      : T -> T;                                                (* Vertex permutation *)
4   e      : T -> T;                                                (* Edge permutation *)
5   boundary : 'I_(S k) -> T;                                       (* Root and boundary darts *)
6   boundary_inj : injective boundary;                               (* Exactly k boundary darts *)
7   vboundary : forall i, v (boundary i) = (boundary i);           (* Boundaries are univalent *)
8   vtrivalent : forall x, v x = x ->                                (* Only root and boundaries *)
9                 exists i, x = boundary i;                         (* are univalent *)
10  v3      : forall x, v (v (v x)) = x;                             (* Vertices: degree 1 or 3 *)
11  eall    : forall x, e x <> x;                                     (* Every dart is in an edge *)
12  e2      : forall x, e (e x) = x;                                 (* Edges have degree 2 *)
13  trans   : transitive v e (boundary ord0);                       (* Darts are transitive *)
14 }.

```

The map is defined as a collection of objects. We are first given T , the type of darts, along with the permutations v and e .

We are then given the root dart and the k boundary darts as a function (named *boundary*) from $[0, \dots, k]$ to our type T , where *boundary*(j) is the j th boundary, and the input 0 corresponds to the root dart. We require this function to be injective through the field *boundary_inj* so that each boundary is distinct.

As boundary darts correspond to univalent vertices, we require that they are fixed points of v through the field *vboundary*.

As non-boundary darts correspond to trivalent vertices, we require that they are not fixed points of v through the field *vtrivalent*. We do this by enforcing the contrapositive; rather than stating that every non-boundary dart is a fixed point, we state that every fixed point is a boundary dart.

We require that v^3 is the identity through the field *v3*. This ensures that every vertex is either univalent or trivalent.

We require that e has no fixed points and is an involution through the fields *eall* and *e2*, respectively.

We require that every dart is reachable through v and e from the root dart through the field *trans*.

This is a valid representation as it fulfills every statement in the definition of a rooted trivalent map. Furthermore, every rooted trivalent map has only one representation up to labeling the darts with some type T . This choice of T , however, warrants for us to define a way to show two rooted trivalent maps are isomorphic.

Note that in any following code, any field F of a rooted trivalent map A with parameter k will be referred to as $F\ k\ A$.

3.4 Isomorphisms of Rooted Trivalent Maps

Isomorphism for rooted trivalent maps formalizes the intuitive notion that two maps are the same up to renaming of darts while preserving the combinatorial structure: vertices, edges (as pairs of darts), the local cyclic order at each vertex, and the boundary vertices. Concretely, an isomorphism is a bijection on the set of darts that transports the incidence relations and the involution defining edge-pairs, and that maps corresponding boundaries to each other.

Source Code 5: Isomorphisms of Rooted Trivalent Maps

```

1 Record isomorphism {k l} (A : rtmap k) (B : rtmap l) := Isomorphism {
2   phi    :> T k A -> T l B;
3   keq    : k = l;
4   phiV   : forall x, phi (v k A x) = v l B (phi x);
5   phiE   : forall x, phi (e k A x) = e l B (phi x);
6   phiB   : forall i, phi (boundary k A i) = boundary l B (cast_ord (f_equal S keq) i)
7 }.
8 Definition isomorphic {k l} (A : rtmap k) (B : rtmap l) := inhabited (isomorphism A B).

```

Step-by-step explanation of the record fields:

1. *phi* is the function from one set of darts to the other.
2. *keq* assures that the maps have the same number of boundaries
3. *phiV* states that going through *v* in the first map and then *phi* is equivalent to taking *phi* and then *v* in the second map.
4. *phiE* states that going through *e* in the first map and then *phi* is equivalent to taking *phi* and then *e* in the second map.
5. *phiB* states that *phi* must map a boundary dart in the first map to the corresponding boundary dart in the second map.

For proofs about isomorphisms (for instance, showing composition of isomorphisms yields an isomorphism) the primary strategy is to compose the underlying *phi* functions and prove that each preservation property composes accordingly. This Rocq record style cleanly isolates the algebraic content (the function *phi*) from the structural lemmas, making subsequent proofs modular and straightforward to script in the proof assistant.

Like other relations we have defined, we wish to prove that *isomorphic* is a reflexive and transitive relation.

Source Code 6: Reflexivity and Transitivity of Isomorphisms

```

1 Lemma isorefl {k} (A : rtmap k) : isomorphic A A.
2 Proof. apply inhabits. refine {|phi:=id;keq:=erefl|}. done. done.
3 move => i. by rewrite cast_ord_id.
4 Qed.
5
6 Lemma isotrans {k l m} (A : rtmap k) (B : rtmap l) (C : rtmap m):
7   isomorphic A B -> isomorphic B C -> isomorphic A C.

```

Reflexivity and transitivity are straightforward to implement and have standard proofs. However, proving symmetry would require for us to be able to find an inverse to *phi*. While it is possible to prove that *phi* is bijective by using the other hypotheses, this was deemed unnecessary for the project.

4 Formalizing Coercions From Linear Lambda Terms to Rooted Trivalent Maps

In order to coerce linear lambda terms into rooted trivalent maps, we first define inductive procedures on rooted trivalent maps in order to perform the application and abstraction operations directly on the rooted trivalent maps.

4.1 Variables

When inducting on a lambda term, the variable case serves as the base case of the induction. Since it does not consist of an operation, all we had to do was define the rooted trivalent map that corresponds to a single variable. As should be expected, the corresponding rooted trivalent map is the trivial map. The root represents the output, and the other boundary represents the free variable input.

Source Code 7: Trivial Rooted Trivalent Map

```

1 Program Definition rtVar : rtmap 1 :=
2 {}
3   T                := (unit + unit) %type;
4   v                := id;
5   e                := (fun x =>
6     match x with
7     | inl tt => inr tt
8     | inr tt => inl tt
9     end);
10  boundary          := (fun a =>
11    match a with
12    | Ordinal 0 _ => inl tt
13    | _Ordinal1  => inr tt
14    end);
15  boundary_inj      := _;
16  vboundary         := _;
17  vtrivalent        := _;
18  v3                := _;
19  eall              := _;
20  e2                := _;
21  trans             := _;
22 {}.
```

As you can see in the code above and in many codes blocks in the future, we make use of program definitions rather than definitions, which allow us to provide only the constructive parts of a rooted trivalent map (the dart type T , the permutations v and e , and the *boundary* function), whereas the proofs (from *boundary_inj* to *trans*) are provided later through the interactive proof environment.

In this case, defining the trivial map is simple.

As you can see in figure 4, there are two darts and one non-root boundary dart. Both darts are fixed by v . The darts are transposed by e . The root is the first boundary and the non-root boundary is the second dart.

As this definition is quite simple and there are only two elements in the dart type, all of the proofs follow from a simple case analysis.

4.2 Application of two Rooted Trivalent Maps

This is the part of the thesis where proofs started to become incredibly challenging. The operation `rtApp` below constructs the rooted trivalent map that represents the application of two lambda subterms. Intuitively, given two rooted trivalent maps M_1 and M_2 representing the function and the argument respectively, `rtApp` creates a new trivalent vertex and connects one incident dart of that vertex to the root dart of M_1 , a second incident dart to the root dart of M_2 , and exposes the third incident dart as the new root of the combined map. In other words, we join the two maps at a new central trivalent vertex whose three half-edges are dedicated to (1) the function submap, (2) the argument submap, and (3) the continuation (the root of the application).

This construction can be pictured as two maps connected to a new trivalent vertex:

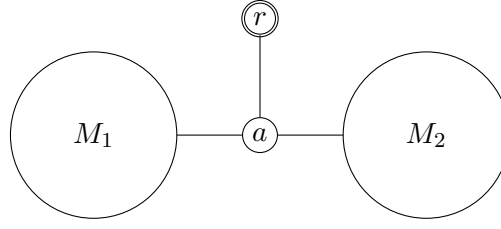


Figure 5: Schematic view of the application operation joining two maps.

Source Code 8: Application of two Rooted Trivalent Maps

```

1 Program Definition rtApp {k j} (A : rtmap k) (B : rtmap j) : rtmap (k+j) :=
2 {}
3   T      := (unit + (unit + (A.(T k) + B.(T j)))) %type;
4   v      := (fun x =>
5     match x with
6     (* Root maps to itself *)
7     | inl tt      => inl tt
8     (* New dart maps to root of A *)
9     | inr (inl tt) => inr (inr (inl (root A)))
10    | inr (inr (inl y)) =>
11      (* Root of A maps to root of B *)
12      if y == root A then inr (inr (inr (root B)))
13      (* Everything else maps as usual *)
14      else inr (inr (inl (A.(v k) y)))
15    | inr (inr (inr z)) =>
16      (* Root of B maps to the new dart *)
17      if z == root B then inr (inl tt)
18      (* Everything else maps as usual *)
19      else inr (inr (inr (B.(v j) z)))
20    end);
21   e      := (fun x =>
22     match x with
23     (* Root maps to new dart *)
24     | inl tt      => inr (inl tt)
25     (* New dart maps to new root *)
26     | inr (inl tt) => inl tt
27     (* Otherwise, map to same place as before *)
28     | inr (inr (inl y)) => inr (inr (inl (A.(e k) y)))
29     | inr (inr (inr z)) => inr (inr (inr (B.(e j) z)))
30     end);
31   boundary := (fun a =>
32     match unlift ord0 a with
33     | None      => inl tt
34     | Some b    => match split b with
35     | inl iA      => inr (inr (inl (A.(boundary k) (lift ord0 iA))))
36     | inr iB      => inr (inr (inr (B.(boundary j) (lift ord0 iB))))
37     end

```

```

38   end);
39   boundary_inj := _;
40   vboundary    := _;
41   vtrivalent   := _;
42   v3           := _;
43   eall         := _;
44   e2           := _;
45   trans        := _;
46 }.
```

The code above shows the natural formalization of the lambda application procedure. We add two darts, the first being the root and the second being the dart adjacent to the root. This second dart creates a new vertex with the roots of the two rooted trivalent maps. We reflect this behavior in v : it takes the new root to itself, the dart next to the root to the root of the first map, the root of the first map to the root of the second map, the root of the second map to the dart incident to the new root, and everything else is mapped as usual. Similarly, for e , the new root and the new dart are transposed, and everything else maps as usual.

The *boundary* function is slightly more complicated. The *unlift* function just serves as a way to say that if the input is 0, we return the new root, and otherwise we distinguish if the index belongs to the first map or the second map and return the corresponding boundary.

The proofs *boundary_inj*, *vboundary*, *vtrivalent*, *v3*, *eall*, and *e2* all follow from an in-depth case analysis.

Meanwhile, the transitivity proof is a bit more complicated. In fact, this was the reason that the current definition for rooted trivalent maps takes an arbitrary dart type T rather than constructing the dart type from the number of boundarys and the number of vertices. The latter was implemented first but made for much more difficult proofs than the current representation.

4.3 Abstraction of a Rooted Trivalent Map

The `rtAbs` operation corresponds to the lambda abstraction constructor. Given a rooted trivalent map M representing the body of an abstraction and given a boundary to treat as the abstraction variable, `rtAbs` attaches a new trivalent vertex adjacent to the root so that:

- one incident dart of the new vertex becomes the root of the resulting map,
- a second incident dart is glued to the previous root of M (so the body becomes a submap reachable from the abstraction vertex), and
- the third incident dart is the abstracted variable occurrence in M .

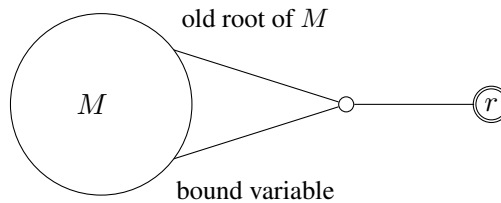


Figure 6: The abstraction operation `rtAbs`. A new trivalent vertex is introduced. One edge becomes the new root r , while the other two edges come from the map M : one is the root of M and one is the bound variable occurrence.

Source Code 9: Abstraction of a Rooted Trivalent Map

```

1 Program Definition rtAbs {k} (b : 'I_(S k)) (A : rtmap (S k)) : rtmap k :=
2 let removed_boundary := boundary (S k) A (lift ord0 b) in
3 {}
4   T      := (unit + (unit + A.(T (S k)))) %type;
5   v      := (fun x =>
6     match x with
7       (* New root maps to itself *)
8       | inl tt      => inl tt
9       (* New dart maps to the previous root *)
10      | inr (inl tt) => inr (inr (root A))
11      | inr (inr y)  =>
12        (* Previous root maps to removed boundary *)
13        if y == root A      then inr (inr removed_boundary) else
14        (* Removed boundary maps to new dart *)
15        if y == removed_boundary then inr (inl tt) else
16        (* Everything else maps as usual *)
17        inr (inr (v (S k) A y))
18    end);
19   e      := (fun x =>
20     match x with
21       (* New root maps to new dart *)
22       | inl tt      => inr (inl tt)
23       (* New dart maps to new root *)
24       | inr (inl tt) => inl tt
25       (* Otherwise, map to same place as before *)
26       | inr (inr y)  => inr (inr (e (S k) A y))
27     end);
28   boundary      := (fun a =>
29     if a == ord0 then inl tt else
30     inr (inr (A.(boundary (S k)) (lift (lift ord0 b) a))));
31   boundary_inj  := _;
32   vboundary     := _;
33   vtrivalent    := _;
34   v3            := _;
35   eall          := _;
36   e2           := _;
37   trans        := _;
38 {}.
```

Like in the application case, the set of darts is the previous set of darts plus the new root and the dart next to the root. The dart next to the root still maps through v to the previous root, but the previous root now maps to the removed boundary dart, which itself maps to the dart next to the root. The permutation e is defined exactly the same as in the abstraction case. The *boundary* function is slightly different. It again maps 0 to the root. Everything else turns into the corresponding boundary in the previous map, skipping over the removed boundary.

Otherwise, the proofs are very similar to the *rtApp* case.

4.4 Final Construction

Now that we have defined the base case $rtVar$ and the procedures $rtApp$ and $rtAbs$, we are able to define our function $llambda_to_rtmap$ which takes a linear lambda term to the corresponding rooted trivalent map.

Source Code 10: Final Construction

```

1 Fixpoint llambda_to_rtmap {k} (term : llambda k) : rtmap k :=
2 match term with
3 | LVar          => rtVar
4 | LApp p q P Q  => rtApp  (llambda_to_rtmap P) (llambda_to_rtmap Q)
5 | LAbs k a P    => rtAbs a (llambda_to_rtmap P)
6 end
7 .

```

The function takes a lambda variable to the trivial map, an application $P(Q)$ to the map formed by applying $rtApp$ to $llambda_to_rtmap(P)$ and $llambda_to_rtmap(Q)$, and it takes a lambda abstraction of P on boundary a to the map formed by applying $rtAbs$ to $llambda_to_rtmap(P)$ on boundary a .

5 Formalizing Coercions From Rooted Trivalent Maps to Linear Lambda Terms

To coerce rooted trivalent maps to linear lambda terms, we must first prove that a rooted trivalent map must have at least two darts. We then state that if it has exactly two darts, it is isomorphic to the trivial map. If it has more than two darts, the vertex adjacent to the root is trivalent. If removing this vertex and the root disconnects the graph, then the vertex corresponds to an application node, and otherwise it corresponds to an abstraction node.

5.1 Proving Maps have at least two Darts

```

1 Lemma Tgeq2 {k} : forall (A : rtmap k),
2   #|T k A| >= 2.

```

This proof was straightforward. A rooted trivalent map must have a root, so it has at least one dart. Passing the root through e gives us another dart, as e has no fixed points. Therefore, a rooted trivalent map must have at least two darts.

5.2 Maps with two Darts are isomorphic to the trivial map

The first step to proving this is to prove that the two darts are the root, which will henceforth be called x , and $e(x)$, which will henceforth be called y .

Source Code 11: All Darts are are x or y

```

1 Lemma Teq2_enum {k} : forall (A : rtmap k), #|T k A| = 2 ->
2   forall (z : T k A), z = x \ / z = y.

```

Even though this statement is obvious mathematically, it takes quite a lot of work to prove it formally in Rocq. To complete the proof, we first assume that $z \neq x$. We then say that the set $T - \{x\}$ has exactly one element and contains both z and y , since we have that $z \neq x$ and $y \neq x$. Therefore, we deduce that $z = y$, as needed.

We next prove that the vertex containing y is univalent.

Source Code 12: y is univalent

```
1 Lemma Teq2_vyeqy {k} : forall (A : rtmap k), #|T k A| = 2 ->
2   v k A y = y.
```

We use the previous lemma to do a case analysis on $v(y)$. If $v(y) = y$ we are done. If $v(y) = x$, we deduce that $v^3(y) = v^2(x)$. Since $v^3 = id$ and $v(x) = x$, we deduce that $y = x$, which is a contradiction. This concludes the proof.

We next prove that the number of non-root boundary vertices must be 1.

Source Code 13: There is exactly one non-root boundary

```
1 Lemma Teq2_keq1 {k} : forall (A : rtmap k),
2   #|T k A| = 2 -> k = 1.
```

We prove this lemma by contradiction. We first deal with the $k = 0$ case. If $k = 0$, there is only one boundary: the root. We use the previous lemma and the trivalent hypothesis from *rtmap* definition to deduce that y is a boundary. Since there is only one boundary, we deduce that $y = x$, which is a contradiction.

We now assume $k \geq 2$. We know that T contains both the root and every boundary. Therefore, $|T| \geq 3$, which is a contradiction.

We deduce that $k = 1$, as needed.

Now we are ready to prove that any map with $|T| = 2$ is isomorphic to the trivial map.

Source Code 14: Final Proof

```
1 Lemma Teq2_Atriv {k} (A : rtmap k) (ET : #|T k A| = 2) :
2   isomorphic A rtVar.
```

Recall that *rtVar* was defined above to be the rooted trivalent map corresponding to a variable. The key part of the proof is the function:

```
1 phi : T 1 A -> T 1 rtVar := (fun z => if z==x then inl tt else inr tt).
```

This function maps the root of A to the root of *rtVar* and anything else to the second dart. Again, since the darts only have two possible values, the other parts of the isomorphism proof are quite simple and follow from a basic case analysis on the darts.

5.3 Maps with more than two Darts

We now consider maps with more than two darts. We first prove that the vertex adjacent to the root is trivalent, as otherwise we would be in the trivial map case. We continue the convention that the root dart is x and the dart adjacent to the root is y .

Source Code 15: y is not a boundary

```

1 Lemma Tgt2_vyneqy {k} : forall (A : rtmap k), #|T k A| > 2 ->
2    $\forall k A y \neq y.$ 

```

To complete this proof, we assume for the sake of contradiction that y is fixed by v . We then prove that, since $|T| > 2$, there must be a third dart z which is not equal to x nor y . We then call upon transitivity to say that z must be reachable from x . Through some case analysis on the reachability, you are able to deduce a contradiction, since it is clear that only x and y are reachable from x .

We now introduce two more darts: $rA = v(y)$ and $rB = v(rA)$, and prove that the four darts x, y, rA, rB are all distinct, which mostly follows from previous lemmas.

Source Code 16: x, y, rA , and rB are pairwise distinct

```

1 Lemma Tgt2_four {k} : forall (A : rtmap k), #|T k A| > 2 ->
2   ( $x \neq y \wedge x \neq rA \wedge x \neq rB \wedge$ 
3      $y \neq rA \wedge y \neq rB \wedge$ 
4      $rA \neq rB$ ).

```

Our goal is now to remove x and y to see if the map remains connected. To this end, we define the new set of darts $preT$ as follows:

```

1  $preT := [\text{set } z : T \text{ k A} \mid (z \neq x) \wedge (z \neq y)]$ 

```

We also prove a couple of minor lemmas concerning the set $preT$, such as the following:

Source Code 17: $z \in preT \implies e(z) \in preT$

```

1 Lemma zIez {k} (A : rtmap k) :
2   forall z, z \in preT -> e k A z \in preT.

```

The proof is rather straightforward, since $e(z) = x \implies z = y$ and $e(z) = y \implies z = x$.

5.4 A Slight Adaptation of Reachability

We now wish to define a version of reachability that does not pass through the vertex adjacent to the root dart. This will be used to see if the map remains connected after removing the root and the vertex adjacent to the root. One might think that we could just use the existing reachability definition on the set $preT$. However, the functions v and e are defined on the set T , so we need some other way to understand this kind of reachability.

Source Code 18: Proposition $preT$ Reachability

```

1 Inductive r_prop {T : finType} (z : T) (v e : T -> T) (root : T) : T -> Prop :=
2   | r_r : (r_prop z v e root) root
3   | r_v : forall x, z \neq x -> z \neq v x -> z \neq v (v x) ->
4     (r_prop z v e root) x -> (r_prop z v e root) (v x)
5   | r_e : forall x, (r_prop z v e root) x -> (r_prop z v e root) (e x).

```

This definition is almost identical to the previous definition of reachability, with the only difference being that when we do a step through v , we assure that we are not currently on the vertex adjacent to the root; that is, that the dart passing through v is neither y , rA , nor rB . Note that we actually allow any dart to be input as the dart to avoid passing through, but for the purposes of this thesis, r_prop will only be called with the first argument being the dart adjacent to the root.

Like our previous definition of reachability, we prove that $preT$ reachability is transitive.

Source Code 19: Transitivity of Proposition $preT$ Reachability

```
1 Lemma r_prop_trans {T : finType} (b z : T) (v e : T -> T) (x y : T) :
2   r_prop z v e x b -> r_prop z v e b y -> r_prop z v e x y.
```

Although this definition is very useful when taking care of the connected case, Rocq does not allow case analyses on propositions when defining objects of a type, and we will need to perform a case analysis on reachability to decide whether we are in the application case or the abstraction case. Therefore, we would like to create a boolean equivalent to the current reachability. This is found in the definition below:

Source Code 20: Boolean $preT$ Reachability

```
1 Definition step_avoid {k A} (z u w : T k A) : bool :=
2   ((w == v k A u) && (w != z) && (v k A w != z) && (u != z)) || (w == e k A u).
3
4 Definition r_bool {k A} (z u w : T k A) : bool :=
5   connect (step_avoid z) u w.
```

We are allowed to define this as a boolean value since we ensure that T is a finite type, which makes any proposition on its members computable. Here, $step_avoid$ describes the same steps allowed in r_prop : either a step through e or a step through v that avoids the vertex adjacent to the root. Note again that we actually allow any dart to be input as the dart to avoid passing through, but for the purposes of this thesis, it will only be called with the first argument being the dart adjacent to the root.

The connect predicate says that through some amount of steps through the function $(step_avoid\ z)$, we can reach w from u .

The connect predicate gives us reflexivity and transitivity for free, but we still wish to prove symmetry.

Source Code 21: Symmetry of Boolean $preT$ Reachability

```
1 Lemma r_bool_sym {k A u w} (z : T k A) :
2   r_bool w u -> r_bool z u w.
```

The proof again comes from the fields $v3$ and $e2$ from the definition of a rooted trivalent map.

Of course, to use the two version of $preT$ reachability interchangeably, we have to prove that they are equivalent.

Source Code 22: Equivalence of Proposition and Boolean $preT$ Reachability

```
1 Lemma bool_prop {k A} (z u w : T k A) :
2   r_bool z u w -> r_prop z (v k A) (e k A) u w.
3
4 Lemma prop_bool {k A} (z u w : T k A) :
5   r_prop z (v k A) (e k A) u w -> r_bool z u w.
```

The proof of the first lemma comes from inducting on the right side of the path that connects u and w in r_bool and performing the same last step in r_prop . The proof of the second lemma comes from doing an induction on r_prop , using the results of the first lemma on the induction hypothesis, and performing the last step manually.

Now that we have proven both forms of $preT$ reachability are equivalent, we are ready to use them to distinguish which of the two cases we are in.

5.5 Abstraction Case

We now assume using proposition reachability that rA and rB are connected in $preT$. We know that this means we are in the abstraction case, and we define the rooted trivalent map obtained by removing x and y . We introduce rAT and rBT , which are just rA and rB , respectively, combined with a proof that they are contained in $preT$. They are created using the Sub function, which takes an element of T and a proof that it is in $preT$ to create an object inside $preT$. Similarly, $insubd$ is a function that returns the second argument if it is contained within $preT$, and otherwise returns the first argument. We use it here even though the second argument is always inside $preT$ in order to provide the proof of membership in the interactive proof environment.

Source Code 23: Rooted Trivalent Map on $preT$

```

1 Program Definition Tgt2_conn {k} (A : rtmap k) (ET : #|T k A| > 2)
2 (EAB : r_prop y v e rA rB) : rtmap (S k) :=
3   let rAT := Sub rA _ : {x | x \in preT} in
4   let rBT := Sub rB _ : {x | x \in preT} in
5   {}
6   T := {x | x \in preT};
7   v := fun d => if d == rAT then rAT else
8             if d == rBT then rBT else
9             insubd rAT (v k A (val d));
10  e := fun d => insubd rAT (e k A (val d));
11  boundary := fun i : 'I_(k.+2) =>
12             if i == ord0 then rAT else
13             match unlift ord_max i with
14             | None    => rBT
15             | Some j  => insubd rAT (boundary k A j)
16             end;
17  {}.
```

This procedure is the inverse procedure of when we defined the $rtAbs$ procedure. Instead of making the roots connected to each other, we make the roots connect back to themselves through v and map everything else as usual.

The boundary function is again a little bit complicated. In the end, the logic is equivalent to mapping 0 to rAT , $k + 1$ to rBT , and everything in between to the same boundary it mapped to beforehand.

Because each $preT$ object now requires a proof that it is inside $preT$, the proofs required a lot more work. The transitivity proof was especially complicated as it required to transfer from the r_prop predicate on the darts in T to prove something about the *reachable* predicate on the darts in $preT$. Otherwise, the proofs are quite similar to those of the $rtAbs$ procedure.

Now that we have proven that this is a rooted trivalent map, it still remains to prove that applying the abstraction procedure on it gives us back the original map.

```

1 Lemma rtmap_decompose {k} (A : rtmap k) (ET : #|T k A| > 2) (EAB : r_prop y v e rA rB) :
2   isomorphic A (rtAbs ord_max (Tgt2_conn A ET EAB))

```

Note that in the original code, this was not a separate lemma but actually one of the goals created by the lemma `rtmap_decompose`.

This proof starts by defining:

```

1 phi : T k A -> T k (rtAbs ord_max (Tgt2_conn A ET EAB)) := fun z =>
2   if z == x then inl tt else if z == y then inr (inl tt) else
3   inr (inr (insubd rAT z)).

```

The remainder of the proof is long and tedious, but the result is eventually proven after lots of case analyses on the inputs of the functions.

5.6 Partitioning T for the Application Case

Now that we are finished proving the Abstraction case, we wish to take care of the disconnected case. To this end, we want to partition the darts in T into those connected to rA , which we will call $setA$, and those connected to rB , which we will call $setB$.

Source Code 24: Defining $setA$ and $setB$

```

1 Definition setA := [set z : T k A | r_bool y rA z ].
2 Definition setB := [set z : T k A | r_bool y rB z ].

```

We also need to prove that these sets actually form a partition. That is, we have to prove that every element in T (other than x and y) is either in $setA$ or $setB$, and that these sets are disjoint.

Source Code 25: Proving $setA$ and $setB$ form a Partition

```

1 Lemma partition (z : T k A) : (z == x) || (z == y) || (z \in setA) || (z \in setB).
2
3 Lemma partition_disjoint : (r_bool y rA rB = false) -> [disjoint setA & setB].

```

The first proof is quite technical but boils down to showing that if z is not in $setA$ nor $setB$, then it cannot be reachable, which contradicts the hypothesis *trans*.

The second proof is much simpler, and shows that if $setA$ and $setB$ were not disjoint, then rA and rB would be connected through transitivity, which is a contradiction.

Partitioning the darts is nice, but it is not enough, as we also need to partition the boundary darts in order to define the *boundary* function for our two new rooted trivalent maps. Therefore, we repeat the process above but for boundary darts specifically.

Source Code 26: Partitioning Boundary Darts

```

1 Definition setAb := [set i : 'I_(S k) | (i != ord0) && r_bool y rA (boundary k A i) ].
2 Definition setBb := [set i : 'I_(S k) | (i != ord0) && r_bool y rB (boundary k A i) ].
3

```

```

4 Lemma partition_boundary (i: 'I_(S k)) (ET : #|T k A| > 2) :
5   (i == ord0) || (i \in setAb) || (i \in setBb).
6
7 Lemma partition_boundary_disjoint: (r_bool y rA rB = false) ->
8   [disjoint setAb & setBb].

```

The proofs for these lemmas were essentially the same as the proofs above.

Unfortunately, at this point in the thesis, we ran out of time. The plan for the rest of the code was to define two new rooted trivalent maps on dart sets *setA* and *setB*, respectively, that have boundary darts *setAb* and *setBb*, respectively, and to show that performing the *rtApp* procedure on these two maps returns the original map. Using this, we could define a coercion procedure *rtmap_to_llambda* that inducts on $|T|$. If $|T| = 2$, it would return a variable, and otherwise it would do a case analysis on *preT* reachability of *rB* from *rA*. If reachability holds, you would inductively call *rtmap_to_llambda* on the map defined on *preT* and return the lambda abstraction of that lambda term. If reachability does not hold, you would inductively call *rtmap_to_llambda* on the two maps defined on *setA* and *setB* and return the lambda application of the two returned lambda terms.

Once *rtmap_to_llambda* was defined, the goal was to prove that it was the inverse function to the *llambda_to_rtmap* coercion. That is, we wished to prove the lemmas:

```

1 Lemma injective {k} : cancel (llambda_to_rtmap (k:=k)) (rtmap_to_llambda (k:=k)).
2
3 Lemma surjective {k} : cancel (rtmap_to_llambda (k:=k)) (llambda_to_rtmap (k:=k)).

```

6 Conclusion

This work produces a mechanically checked foundation for the combinatorial correspondence between arity-indexed open linear lambda terms and rooted trivalent maps. Concretely, the development delivers (1) an inductive type of linear lambda terms indexed by the number of free variables; (2) a permutation-based record for rooted trivalent maps that exposes darts, a vertex permutation, an edge involution, an indexed boundary function and a reachability/transitivity invariant; (3) executable map constructors corresponding to the term constructors ("rtVar", "rtApp", "rtAbs"); and (4) an isomorphism record that isolates renaming and supports transport of structural facts between maps. These components are organized so that the principal combinatorial invariants and decision procedures are available as executable definitions suitable for further mechanized reasoning.

The mechanization verifies a broad collection of routine but essential lemmas: degree and boundary facts for small maps, reflexivity/transitivity (and symmetry under the map axioms) of reachability, correctness properties of the primitive constructors, and decomposition lemmas that detect whether the vertex adjacent to the root represents an abstraction or an application by testing connectivity after removing the root and its neighbor. The proofs make systematic use of a booleanized reachability predicate that permits decidable case splits and of small transfer lemmas to relate propositions about the full dart set to facts about sub-dart types.

The principal remaining objective is the formal construction of the inverse coercion and the two cancellation theorems. The outstanding obligations are precise and finite:

1. complete the application case partition;
2. implement the submap constructors on those partitioned dart sets and prove the standard *rtmap* obligations for each submap;

3. prove that re-gluing the two submaps by `rtApp` reconstructs the original map up to isomorphism;
4. define `rtmap_to_llambda` by induction on the number of darts, using the booleanized reachability test to distinguish abstraction from application; and
5. prove the cancellation lemmas (injectivity and surjectivity) by structural induction.

Completing these steps will yield a fully machine-checked bijectivity theorem: fully executable coercions `llambda_to_rtmap` and `rtmap_to_llambda` together with formal proofs that they cancel each other. The finished development will make the informal combinatorial argument fully rigorous in the chosen formalization, provide a reusable library of combinatorial lemmas for maps and terms, and serve as a stable basis for further mechanized work.

7 References

- [1] Assia Mahboubi and Enrico Tassi. *Mathematical Components*. Zenodo, 2025. Version 2.5.0; used for formalizing mathematics in the Rocq Prover.
- [2] The Rocq Development Team. *The Rocq Prover, version 8.20.1*, 2026. Formerly known as the Coq Proof Assistant. Available at <https://rocq-prover.org/>.
- [3] Noam Zeilberger. Linear lambda terms as invariants of rooted trivalent maps. *Journal of Functional Programming*, 26:e21, 2016.

A Appendix

The full code is available on [github](#). Note that some variables were renamed in the paper, but otherwise the general structure of the code is the same.